

2^{do} Parcial - Técnica y Diseño de Algoritmos (ex Algo3)
17 de Nov. 2025 - 2^{do} Cuatrimestre

Nombre y Apellido	#Orden	L.U.	Turno	Corrector	Nota
				Dafne Y.	
			Ej 1	Ej 2	Ej 3
			Ej 4		
Puntajes parciales					

Duración: 4 horas. Este examen es individual y a libro cerrado. La nota de aprobación es de 60/100 puntos

Ej 1: (16 Puntos) Marque la única opción correcta. Puntajes: **Correcto: 4 pts | Incorrecto: -2 pt**

I. Dado un grafo G de $n \geq 2$ nodos sin nodos de grado 0, ¿cuáles de los siguientes items son condiciones suficientes para que G sea árbol? A) G tiene exactamente 2 nodos de grado 1 y $n - 1$ aristas B) G tiene exactamente 2 nodos de grado 1 y sin ciclos C) G tiene exactamente 2 nodos de grado 1 y es conexo D) G tiene exactamente 2 nodos de grado 1 y el resto de grado 2	☒	III. Sea F el valor de un flujo en una red $G = (N, A)$ con la función de capacidad u definida en A y S un corte ($S \subseteq N$ y $S \cap \{s, t\} = \{s\}$, siendo s y t fuente y sumidero de G respectivamente), entonces A) $F = u(S)$ B) $F \geq u(S)$ C) $F \leq u(S)$ D) $F \neq u(S)$	☒
	☒	IV. Un grafo G es autocomplementario si es isomorfo a su grafo complemento. La cantidad de grafos de esta clase con entre 1 y 7 nodos es A) 1 B) 2 C) 3 D) 4 o más	☒
II. Para determinar un ciclo con menor cantidad de aristas de un grafo conexo $G = (V, E)$, es posible hallarlo mediante: A) una única aplicación de DFS B) una única aplicación de BFS C) $\Theta(V)$ aplicaciones de DFS, pero no en $O(1)$ aplicaciones D) $\Theta(V)$ aplicaciones de BFS, pero no en $O(1)$ aplicaciones	☒		
	☒		

Ej 2: (28 Puntos) Sea G un grafo. Sean v y w dos nodos de G tal que $G - v$ es conexo, $G - w$ es conexo, y $G - \{v, w\}$ no es conexo.

- Demostrar que existe un ciclo simple (sin nodos ni aristas repetidas) que contiene a v y a w . **Ayuda:** pensar en las componentes conexas de $G - \{v, w\}$. **(18 Puntos)**
- Diseñar un algoritmo eficiente que, dados G , v y w , encuentre un ciclo simple que contenga a v y w . Determine y justifique la complejidad temporal. **(10 Puntos)**

Ej 3: (28 Puntos) En una gran plaza hay n puestos de comida, numerados del 1 al n . Cada puesto vende uno o más de los k ($1 \leq k$) tipos de comida disponibles. En la plaza existen m caminos, que se pueden recorrer en ambos sentidos, y cada camino i conecta dos puestos u_i y v_i ($1 \leq u_i, v_i \leq n$) y se recorre en s_i segundos.

Mao Mao y Luna son dos amigos que actualmente están en el puesto de comida 1, el único que está cerrado hoy y no vende comida. Los amigos quieren recorrer los puestos y reunirse a comer juntos en el puesto n . Pero quieren haber comprado, entre ambos, todos los posibles tipos de comida en su trayecto desde el puesto 1 hasta el puesto n .

Como tienen hambre, quieren hacer eso lo más rápido posible, así que deciden ir por caminos separados (no necesariamente disjuntos) y encontrarse en el puesto n . Van a empezar a comer apenas estén ambos en el puesto n , y quieren empezar a comer lo antes posible. Cuando pasan por un puesto de comida, pueden comprar instantáneamente todos los tipos de comida que vende ese puesto.

Quieren saber, ¿cuál es el tiempo mínimo en segundos que deben esperar hasta que ambos estén en el puesto n habiendo comprado entre los dos todos los tipos de comida?

Tu deber es modelar el problema usando un grafo $G = (V, E)$ y describir los pre- y pos-procesados necesarios, cumpliendo que:

- $(|V| + |E|) \in O(k \cdot 2^k(n + m))$
- Las aristas del grafo G o bien no son ponderadas, o bien tienen pesos no negativos.
- El tiempo de ejecución del pre-procesado (que construye el grafo G) es $\in O(k \cdot 2^k(n + m))$
- El tiempo de ejecución del pos-procesado (que convierte la respuesta del algoritmo de grafos aplicado sobre G en la respuesta del problema original) es $\in O(k^2 \cdot 4^k)$. La mejor complejidad que conocemos es $O(k \cdot 2^k)$
- Aplica un algoritmo de camino mínimo o árbol generador mínimo sobre G que resuelva el problema en la mejor complejidad temporal.

Ej 4: (28 Puntos) Luego del éxito del torneo anterior los organizadores del torneo de voley TYVA se preparan para hacer una nueva edición. Esta vez proponen un nuevo sistema por rondas dónde en cada ronda de n equipos pasan los $n - 1$ equipos con mejor puntaje, el puntaje además se satura en P puntos, es decir una vez que un equipo llega a P puntos cualquier punto extra no cuenta para la clasificación a la siguiente ronda.

Para hacer el torneo más picante, si un equipo ganó demasiado y tiene puntos de sobra puede regalarle algunos de estos a otro equipo pero solo si le ganaron previamente al mismo. Se conoce la lista de partidos L que se jugaron. Para que no sea demasiado desbalanceado además se propone un límite Q de puntos que puede recibir en total un equipo en cada ronda.

El equipo de definitivamente no Echu o como acordaremos a $\neg$ Echu tiene K puntos y por alguna extraña razón los equipos restantes le tomaron odio y no quieren que pase de la primera ronda por lo cual conspiran para redistribuir los puntos sobrantes entre ellos.

$\neg$ Echu hizo los cálculos y vio que si se repartieran los puntos sin ninguna de las restricciones entonces su equipo no pasaría a la siguiente ronda.

- (a) Ayudar a $\neg$ Echu a modelar el problema como un problema de flujo para ver si es posible que su equipo pase, es decir, que no es posible redistribuir los puntajes de forma que todos los equipos tengan más de K puntos sabiendo que $K < P$ y que se concen los puntajes iniciales de cada equipo p_i .
- (b) Justificar por qué el modelo es correcto.
- (c) Calcular su complejidad.

EJ 2 - A:

Sea G un grafo, tenemos:

p : Sean v y w dos nodos de G tq $G-v$ es conexo, $G-w$ es conexo
y $G-\{v, w\}$ no es conexo

q : Existe un ciclo simple que contiene a v y a w

Queremos demostrar $p \Rightarrow q$, lo vamos a hacer por reducción al absurdo $p \wedge \neg q \Rightarrow \text{ABS}$.

Asumimos como verdaderos p y también asumimos que NO existe un ciclo simple que contenga a v y a w ($\neg q$)

Partiendo del grafo G , definimos a $G' = G - v$ y por p sabemos que G' es conexo.

Pero por p también sabemos que $G' - w$ NO es conexo.

Que G' sea conexo y que $G' - w$ no lo sea, nos indica al eliminar a w (y sus aristas incidentes) aumentamos la cantidad de componentes conexas, entonces w tiene al menos una arista incidente en G' tal que esa arista es puente.

Análogamente, se puede hacer el mismo análisis con v y definiendo a $G'' = G - w$.

Por lo que sabemos que tenemos, como mínimo, 2 aristas puente distintas (una por parte de v y otra por parte de w) (es decir, una contiene a v y la otra es incidente en w)

La pregunta que nos hacemos es ¿cómo sabemos que estas, al menos dos puentes, están conectados?

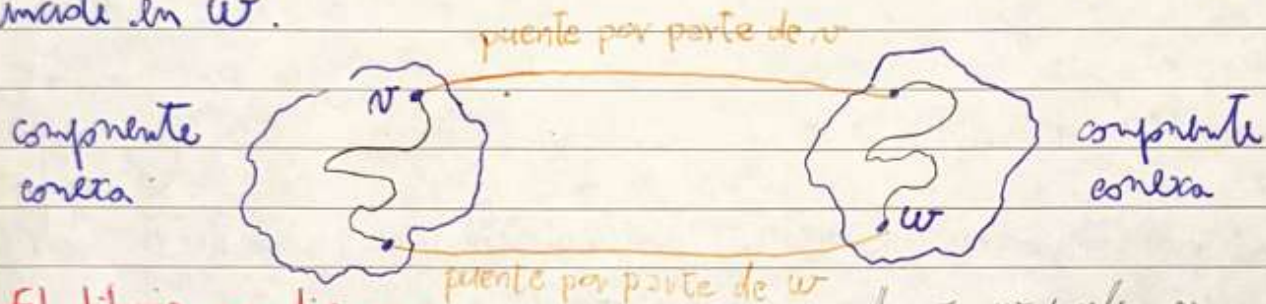
Sabemos que estas puentes unen componentes conexas puesto que G es conexo y $G - \{v, w\}$ no lo es.

Contaej:



Entonces, independientemente del modo al que ^{llegue} el puente que incide en v , como estoy llegando a una componente conexa, por definición de conexo, voy a poder llegar al otro modo donde arranca el otro puente (el puente por parte de w).

Análogamente, puedo hacer el mismo análisis arrancando del puente que incide en w .



El dibujo me dice que no entendiste la propiedad de "articulación" que hay que demostrar.

v y w pueden ser cualquiera de los 4 nodos marcados, siempre que no formen parte del mismo puente.

Esto no implica que tenemos dos caminos distintos porque tenemos, como mínimo, dos puentes distintos (nota que el puente (v, w) no puede existir xq $G-v$ (o $G-w$) no sería conexo, y en caso de que fuera conexo nos indicarían que hay otros puentes que no inciden ni en v ni en w entonces haría que $G-\{v, w\}$ fuera conexo, se nos contradice con p).

Entonces tenemos dos caminos distintos y disjuntos qd. que "comienzan" por la primera componente conexa. no comparte nada con "comienzan" por la segunda componente conexa.

Dos caminos distintos y disjuntos de llegar de v a w , por ende, existe un ciclo simple que contiene a v y a w ... pero esto es ABSURDO porque contradice lo que asumimos $\neg q$.

No termina de cerrar la argumentación. Era una demo más constructiva, no por absurdo.

Por reducción al absurdo, queda demostrado $p \Rightarrow q$.

(R-)

EJ2-B:

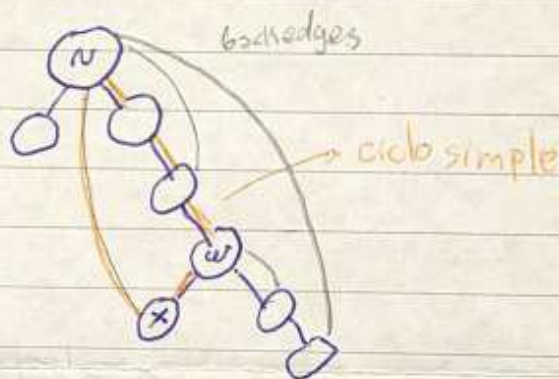
El algoritmo tiene que encontrar el árbol DFS ejecutándose desde el nodo v (en analogía con w) y el ciclo ^{mínimo} que contiene a v y w va a ser la unión de los caminos:

M032 2/3

caminos de w hasta la raíz (o sea, v) $\cup$

caminos de x hasta w , donde x es un nodo que tiene a w como ancestro y tiene una back-edge hacia la raíz (o sea, v) $\cup$

back-edge (x, v)



Algoritmo:

1. Armar el grafo con lista de adyacencia $O(n+m)$
2. Ejecutar DFS desde v (en analogía con w) $O(n+m)$ descendiente
3. Marcar todos los nodos del árbol DFS que tienen como ancestro a w $O(n)$
4. Buscar en los backedges generados por DFS aquellos en los que incide sobre v y sobre algún nodo de los marcados como descendiente de w $O(m)$
5. Devolver en backedge + camino (x, w) + caminos (w, v) $O(n)$

Complejidad final: $O(n+m)$

(B)

EJ 3

Lo voy a modelar con un grafo de estados.

Tengo n puntos, a cada punto le voy a poner un 2^n estados

Imaginemos que tenemos un "array" de longitud n donde voy "guardando" los tipos de comida que ya pude conseguir a medida que voy avanzando en los puntos.

En el punto 1 avanzo con $[0, 0, 0, \dots, 0]$ ya que no tengo ninguna comida, luego es el resto de los nodos va a ser del orden $O(2^n)$ ya que para cada punto tengo que poder admitir un 2^n estados. Por ejemplo, si me muevo del punto 1 al 2 y justo el punto 2 tiene disponibles la variedad 1 y 3 de comida, mi estado de ese nodo va a ser $[1, 0, 1, 0, \dots, 0]$ que quiere decir que conseguí la comida 1 y 3.

Además si punto voy guardando comida de la variedad n

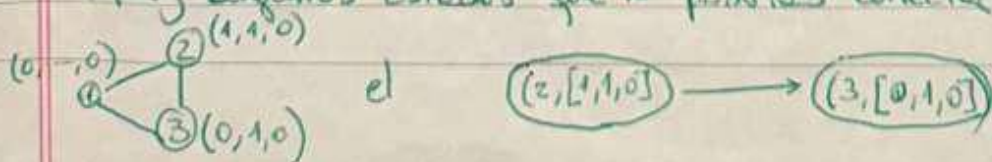
Los aristas van a ser la transición entre estados, junto con el tiempo asociado a hacer la transición. Como por cada estado de cada punto voy a tener que conectarlo con todos los estados de todos los demás puntos y como tengo hasta n variedades voy a tener aristas del orden $O(n \cdot 2^n)$

Nodos: $O(n \cdot 2^n)$

Aristas: $O(m \cdot n \cdot 2^n)$

Está bien pero ¿cuál es la condición para conectar dos nodos con una arista y qué peso le asignas a cada una?

Hay algunos estados que no podrías conectar tipo



con peso 0 en sus orígenes

$O(N)$

Luego, voy a crear n nodos fantasma y los voy a conectar con solo 2 nodos estados y esa conexión solo va a ser la operación lógica XOR entre los estados de entre nodos de todo 1, ejemplo:

$$\begin{array}{r} 101100 \\ \text{xor } 010011 \\ \hline 111111 \end{array}$$

xq son por bits

→ tengo todos los variables

Luego, ejecuto Dijkstra desde el puerto 1 (que solo el puerto 1 tiene 1 estado) para obtener las caminos mínimos desde los puertos a todos los nodos $O(\min\{n^2, (n+m) \cdot \log n, m+n \cdot \log n\})$

→ está en la mano por este problema $(n \cdot 2^{n/2})^2$

Me quedo con la distancia mínima y con la inmediatamente anterior de las distancias a mis n nodos fantasma $O(N)$

Después como resguardo la inmediatamente anterior a mi distancia mínima (la 2° mínima) Me falta la parte de la decisión

C_1 C_2 Comidas

Acá calculo que quisiste unir nodos u_1 y u_2 tales que $C_2 = C_1 \cup u_1$

lo que se venía acumulando

Corrección

corrección: solo conecto a mis nodos fantasma los estados de mi puerto n que den como resultado XOR todos en 1

* En donde se analizan los caminos para Mao Mao y luego y de los que tardan menos cuál es el que tiene todos los comidas → pero el modelo está casi bien

(B)